WEB DEVELOPMENT · 3RD SEMESTER

Week 02 · Block 2b

HTML Forms

<web-development> · Hochschule Ansbach
Matthias Kellnhofer

After this block, you can…

- Build a `<form>` with the right **`action`** and **`method`**
- Use the common **input types** and pair each with a **`<label>`**
- Explain what the browser **sends** on submit — and how it maps to Block 1
- Add **native validation** (`required`, `type`, `pattern`) for better UX
- Say why that validation is **never** a security boundary

Until now, data has flowed one way: **server → you**.

Forms are the way back.

They let the user send data *to* the server — and finally put `POST` to work.

PART 1

The <form> element

A form wraps controls

```
<form action="/register" method="post">
  <label for="email">Email</label>
  <input id="email" name="email" type="email" />
  <button>Sign up</button>
</form>
```

- `<form>` groups **controls** (inputs, selects, buttons…) into one submission
- Pressing the button (or Enter) **submits** everything inside it
- Two attributes decide what happens next: **`action`** and **`method`**

action & method

```
<form action="/register" method="post">
```

- **`action`** — *where* the data goes: the URL (a server endpoint)
- **`method`** — *how* it's sent: **`get`** or **`post`** *(remember these from Block 1?)*

No `method` given? It defaults to **GET**. For anything that creates or changes data, you want **POST**.

GET vs POST — for forms

`method="get"`

- Data is appended to the URL: `/search?q=web`
- Visible & bookmarkable
- For **searches / filters** (reads)

`method="post"`

- Data travels in the request **body**
- Not shown in the URL
- For **sign-ups, logins**, anything with side effects

Same split as Block 1: `GET` reads, `POST` writes. A registration form is a **POST**.

PART 2

Inputs & Labels

One element, many types

The `<input>` element changes behavior based on its **`type`**:

```
<input type="text" />
<input type="email" />
<input type="password" />
<input type="number" />
<input type="date" />
<input type="checkbox" />
```

The `type` sets the on-screen control, the mobile **keyboard**, and the built-in **validation**. Pick the most specific one that fits.

Always label your inputs

```
<label for="email">Email</label>
<input id="email" name="email" type="email" />
```

- The label's `for` matches the input's `id` — that's the link
- Clicking the label **focuses** the input (a bigger, easier target)
- A screen reader announces the label when the field is focused

This is the key form-accessibility win. A `placeholder` is *not* a label — it vanishes as you type and many screen readers skip it. Always use a real `<label>`.

The name attribute

```
<input name="email" type="email" />
```

- On submit, each control is sent as a **`name=value`** pair
- The **`name`** is the **key** the server reads the value by
- **No `name` → not sent at all**

`id` is for *linking and styling*; `name` is for *submitting*. A field usually needs both.

More controls

```
<textarea name="bio"></textarea>
<select name="country">
  <option value="de">Germany</option>
  <option value="at">Austria</option>
</select>
<input type="checkbox" name="terms" />          <!-- on / off -->
<input type="radio" name="plan" value="free" />  <!-- pick one -->
```

- `<textarea>` — multi-line text · `<select>` / `<option>` — a dropdown
- **Radios share a `name`** → the browser lets you pick only one of the group

PART 3

Submitting

What the browser actually sends

```
<form action="/register" method="get">
  <input name="user" value="alex" />
  <input name="age"  value="21" />
</form>
```

On submit, the browser builds a request — **exactly the kind from Block 1**:

```
GET /register?user=alex&age=21 HTTP/1.1
```

- Each control → one `name=value` pair
- **GET** → joined into the URL's **query string** (`?a=1&b=2`)
- **POST** → the same pairs go in the **body** instead

See it yourself — no server needed

You can watch what the form *sends* before we ever build a backend (that's Block 7):

- Set `method="get"`, submit, and read the **URL** — your data is right there in the query string
- Open the **Network tab**, find the request, and inspect its **query string** (GET) or **form data** (POST)

The form generates the request; the server just isn't answering it yet.

PART 4

Validation

Native validation attributes

not examined

The browser can enforce rules **for free** — just add attributes:

```
<input type="email" required />
<input type="text" minlength="3" maxlength="20" required />
<input type="number" min="18" max="120" />
<input type="text" pattern="[A-Za-z]+" />
```

- `required` — can't submit it empty
- `type="email"` — must look like an email
- `min` / `max`, `minlength` / `maxlength`, `pattern` — value constraints

The browser helps the user

When a constraint fails on submit:

- Submission is **blocked**
- A **message** pops up on the offending field
- Focus jumps to it

Good UX, clear feedback, and **zero JavaScript**. This is the first line of help for the *user*.

⚠️ Validation is UX — not security

Native validation runs **in the browser**, so the **user controls it**.

- They can edit the HTML, disable JavaScript, or send a **raw HTTP request** (it's just text — Block 1) that skips the form entirely
- So it stops honest mistakes, not attackers

Never trust data from the client. The server must validate *everything* again — that's the real boundary. We do exactly this in **Block 7a · Web Security I**.

PART 5

Accessible Forms

Group and name your fields

```
<fieldset>
  <legend>Account</legend>

  <label for="email">Email</label>
  <input id="email" name="email" type="email" required />

  <label for="pw">Password</label>
  <input id="pw" name="password" type="password" required />
</fieldset>
```

- `<fieldset>` groups related fields; `<legend>` names the group
- Every input has its own `<label>`
- Mark required fields in **text**, not by colour alone

Recap

- A `<form>` collects controls and sends them to **`action`** using **`method`**
- **GET** → data in the URL; **POST** → data in the body *(Block 1, in action)*
- Every control needs a **`name`** (the key) and a **`<label>`** (accessibility)
- **Native validation** (`required`, `type`, `pattern`) improves **UX** — for free
- …but it's **never security**: the **server** must re-check everything *(Block 7a)*

<EXERCISE> · 45 MIN

Your turn 👉

Build a **registration form** — varied inputs, proper labels, native validation.

1. A `<form>` with several **input types** (text, email, password, number, date, checkbox…)
2. A `<label>` correctly paired with **every** input
3. Native **validation**: `required`, `type="email"`, a `min` / `max` or `pattern`
4. Submit with `method="get"`: find your data in the **URL** and the **Network tab**

At home — Task 6: switch to `method="post"` and find your data in the request body.

Starter: `exercises/week02/2b/` — open `README.md`. One field is done as a pattern; you add the rest.